

هل سيحل الذكاء الاصطناعي في البرمجة محل جميع المبرمجين؟ نظرة إلى الواقع في القطاع

Zhihu • 2026-09-19

منذ أن تحسّنت نتائج البرمجة بالذكاء الاصطناعي تحسّناً كبيراً في نهاية 2025 ومطلع 2026، صار الذكاء الاصطناعي يحل محل جزء كبير من عمل المبرمجين في كتابة الشيفرة، بل معظمه أحياناً. وقد أثارت هذه الظاهرة بين المبرمجين مشاعر الهلع واليأس. وفي الوقت نفسه، استغلت شركات الذكاء الاصطناعي التقدم المحقق في هذا المجال لتضخّم وظائف الذكاء الاصطناعي بلا حدود، وتردد أن جميع ذوي الياقات البيضاء سيفقدون وظائفهم في غضون سنة أو سنتين. وسارعت وسائل الإعلام الذاتية كلها إلى اللحاق بالركب، تنشر الهلع وتضخ القلق.

وفي هذه المسألة، يلزمنا أن نفحص الأمر بجدية. فهل تستطيع البرمجة بالذكاء الاصطناعي أن تحل محل جميع المبرمجين؟ وإن لم تستطع، فما نسبة الاستبدال الفعلية؟

في الواقع، قدّمت في جواب كتبه قبل أشهر قليلة («أول سيف للذكاء الاصطناعي يقع على المبرمجين، فهل سيفقد المبرمجون وظائفهم بأعداد كبيرة؟») تقديري الخاص: لا يستطيع الذكاء الاصطناعي أن يحل محل العمل البشري كله، ونسبة الاستبدال من طرف إلى طرف لا تتجاوز 30%.

حين تدخل منظومات الآلات الذكية التي يكون الذكاء الاصطناعي قلبها الإنتاج على نطاق واسع، سيختفي عمل كتابة الشيفرة يدوياً أساساً. وبالطبع قد يظل تطوير الشيفرة الجوهرية المعقدة بنيوياً أو ذات متطلبات الأداء العالية جدّاً بحاجة إلى تدخل بشري مكثف، أما كتابة الشيفرة العادية الواسعة فسيحل الذكاء الاصطناعي محلها بالكامل. فكم شخصاً سيستبدل تحديداً؟ بحسب النتائج الفعلية لاستخدامي، ووفق ما ردّ به آخرون على Zhihu، يتراوح رفع الكفاءة بين 20% و50% تقريباً. وإذا استقرّت البرمجة بالذكاء الاصطناعي فعلياً (وهي الآن في مرحلة تجربة واسعة في القطاع)، وبحسب تقدير نسبة المطوّرين الصرفين بـ60%، فسيقلّص ذلك نحو 12%-30% من وظائف المبرمجين.

لقد مضى نصف عام على الأقل على استقرار البرمجة بالذكاء الاصطناعي على نطاق واسع، ولا سيما في الصين والولايات المتحدة، حيث تُلزم الشركات الكبرى موظفيها تقريباً باستخدام وكلاء الذكاء الاصطناعي في التطوير. فما النتيجة فعلاً؟

بحسب تقرير ماكنزي^[1] يمكن رفع الإنتاجية بنحو 16%-30%، وهو ما يوافق تقديري السابق في العموم.

McKinsey analyzed nearly 300 publicly traded companies to understand how AI is reshaping software development.^[2] We found that a small group of top performers—roughly the top quintile—are achieving 16–30 percent improvements in productivity, time to market, and customer experience, along with 31–45 percent gains in software quality.

知乎 @最最遥远的路

وفي دراسة حديثة^[2]:

أدوات الذكاء الاصطناعي التوليدي المستخدمة في تطوير البرمجيات تتقدم بسرعة مذهلة؛ فقد انتقلت في سنوات قليلة من الإكمال التلقائي البسيط للشفيرة إلى وكلاء برمجة أذكىء مستقلين تمامًا (coding agents). فهل تترجم هذه التعزيزات في القدرات إلى نمو في الإنتاجية على مستوى المهام في العالم الواقعي؟ وحتى لو ترجمت، فهل يعني تحسّن الكفاءة على مستوى المهام زيادة في الناتج النهائي؟ هذه الأسئلة جوهرية لفهم الأثر الاقتصادي للذكاء الاصطناعي. ومن خلال دراسة أحداث مقترنة بالمطورين، نجد أن الأثر في رفع الإنتاجية على مستوى المهام يزداد وضوحًا مع تعاقب أجيال أدوات البرمجة بالذكاء الاصطناعي: فقد زادت خاصية الإكمال التلقائي حجم الشيفرة المُودّعة أسبوعيًا بنحو 30%، ورفع الوكلاء المتزامنون (sync agents) هذا الأثر التراكمي إلى نحو 180%، ثم رفعه الوكلاء غير المتزامنين (async agents) إلى نحو 240%. غير أن هذه المكاسب تتضاءل بحدّة حين نصعد في المستويات إلى مرحلة الإنتاج: فرغم أن الوكلاء المتزامنين زادوا عدد أسطر الشيفرة بنسبة 958% وطلبات السحب (pull requests) بنسبة 86%، فإن عدد الإصدارات المنشورة في النهاية لم يزد إلا بنسبة 20%. ويُظهر نشاط البرمجة الإجمالي على GitHub القاعدة نفسها.

AI Boosts Coders' Output, But Not Their Releases

TOOL TYPE	LINES OF CODE	COMMITTS	RELEASES
Autocomplete Tools	228%	36%	10%
Sync Agents	741%	109%	20%
Async Agents	658%	34%	n/a

Percentages reflect how much a developer's output rose after adopting that category of AI tool, compared with their own activity before adoption. AI delivers its biggest boost at the code-writing stage, while human-driven steps — reviewing, merging, releasing — absorb most of that advantage before software actually ships.

Source: Adapted from M. Demirel, L. Musolff, and L. Yang, "Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools," 2026.

MIT
MANAGEMENT
SLOAN SCHOOL

المصدر: <https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

وليس البحث وحده هو الذي يؤكد محدودية البرمجة بالذكاء الاصطناعي، بل إن المطورين والمديرين الذين يستخدمون وكلاء الذكاء الاصطناعي في عملهم يلمسون ذلك لمسًا مباشرًا. فقد استفاق المبرمجون من صدمتهم الأولى تجاه البرمجة بالذكاء الاصطناعي، وأخذوا يدركون محدوديتها عبر التطبيق الإنتاجي الفعلي. فمثلًا، كتب Cal Newport (أستاذ علوم الحاسوب في جامعة جورجتاون) مقالًا في مدونته^[3] يحكي فيه عن مطور ارتفعت إنتاجيته عدة أضعاف بعد البرمجة بـ Claude Code فصح «really, really good»، ثم بعد حادثتين في بيئة الإنتاج سببتهما شيفرة الذكاء الاصطناعي، كاد يفقد وظيفته على إثرهما، فخابت آماله في البرمجة بالذكاء الاصطناعي، وعاد إلى كتابة الشيفرة الجوهريّة بيده، وقصر استخدام الذكاء الاصطناعي على كتابة حالات الاختبار والسكريبتات وما شابهها. ويرى هذا المطور أن البرمجة على الطريقة القديمة بطيئة، لكنها موثوقة وجودتها مضمونة. ويقول الكاتب إن هذه الحالة ليست معزولة، بل يعبر كثير من مهندسي القطاع عن رأي مشابه.

فأين تكمن مشكلة البرمجة بالذكاء الاصطناعي؟ كتب أحد المبرمجين في مدونته^[4] عن «تآكل القيود: هشاشة وكلاء LLM في توليد شيفرة الواجهة الخلفية»^[5]:

كشفت دراستنا ظاهرة «تآكل القيود»: فمع ازدياد المتطلبات البنيوية يتراجع أداء الوكلاء تراجعًا واضحًا. فمن المهام المرجعية إلى المهام المحددة تحديدًا كاملاً، انخفض معدل اجتياز التأكيدات في التكوينات

الجيدة الأداء بمقدار 30 نقطة مئوية في المتوسط، وهبط في بعض التكوينات الأضعف أداءً إلى مستوى يقارب الصفر. وقد كشف تحليل الحساسية للإطار اختلافات كبيرة في الأداء: فالوكلاء يبلون بلاءً حسنًا في الأطر البسيطة والصريحة (Flask)، لكن أداءهم المتوسط ينهار في البيئات المعتمدة اعتمادًا شديدًا على الأعراف (Django FastAPI). وأخيرًا، يُظهر تحليل الأخطاء أن عيوب طبقة البيانات (مثل بناء الاستعلامات الخاطئ والاستثناءات وقت التشغيل في ORM) هي المصدر الرئيسي لهذه المشكلات. وتؤكد هذه الدراسة أن تلبية المتطلبات الوظيفية والبنوية في وقت واحد لا تزال تحديًا محوريًا لم يُحل بعد أمام وكلاء البرمجة.

وقد أشار مدير تقني (CTO) في شركة هندية في مقال حديث^[6] إلى الآتي.

فالشفرة التي يكتبها الذكاء الاصطناعي سريعة، لكن بسبب هذه المشكلات المذكورة أعلاه يجد المطورون أنفسهم مضطرين إلى تفحص الشيفرة المولدة بالذكاء الاصطناعي بعناية قبل دمجها في مستودع الشيفرة. وهذا يطرح مسألتين. الأولى أن سرعة تفحص الشيفرة أدنى بكثير من سرعة توليدها، ما يجعل زيادة الكفاءة التي يوفرها الذكاء الاصطناعي تتقلص تقلصًا كبيرًا. والثانية أن المبرمجين الجدد يفتقدون الفرصة لإمساك الشيفرة وكتابتها بأيديهم، فلا تنمو قدراتهم؛ وحين يُسرح الجيل القديم من المبرمجين أو يُحال إلى التقاعد، ستتباطأ سرعة تفحص الشيفرة أكثر، بل لن يبقى التفحص قادرًا على كشف المشكلات.

بل إن بعض الشركات تُعلّق آمالًا كبيرة على البرمجة بالذكاء الاصطناعي، فتضغط فترات التطوير ضغطًا حادًا بحيث لا يجد المبرمجون وقتًا لفحص الشيفرة بجدية. ومن وجهة نظر المطورين، الشركة ليست ملكهم؛ فإن طلب الرئيس الأعلى السرعة، اكتفوا بفحص سطحي أو دمجوها دون فحص أصلاً. وإن ظهرت مشكلة بعد التشغيل، فالأسوأ أن يستقيلوا. وسواء كان الأمر إجهادًا في التفحص أو تفحصًا غير كافٍ، فالنتيجة واحدة: تختبئ في شيفرة بيئة الإنتاج أكوام من الأخطاء والروائح الكريهة في الشيفرة وما شابه، بينما من يتولى صيانة هذه البرمجيات لا يعرف الشيفرة في الحقيقة معرفة جيدة. وكل هذا الدين التقني ينتج ما يسمى «فجوة قابلية الصيانة» (The Maintainability Gap)^[7].

تواصل البرمجة بمساعدة الذكاء الاصطناعي تقدمها السريع، وبلغ حجم الشيفرة التي ينتجها أكثر المطورين حماسًا مستوى قياسيًا غير مسبوق. غير أن تحليل بيانات تغييرات الشيفرة على مدى أربع سنوات يُظهر أن مؤشرات قابلية الصيانة في تراجع: انخفضت الاستدعاءات بين الملفات (وهي تجلّ لإعادة استخدام الشيفرة) بنسبة 35%؛ وتراجعت عمليات نقل أسطر الشيفرة في إعادة الهيكلة بنسبة 70% مقارنة بمستوى 2022؛ كما تراجع العمل على صيانة الشيفرة القديمة العالقة بنسبة 74%. وفي الوقت نفسه، نلاحظ تصاعدًا في ظواهر مقلقة: زاد سلوك «النسخ/اللصق» داخل الإيداع الواحد بنسبة 41%، وتكررت كتل الشيفرة بنسبة 81%، وزادت البنى البرمجية التي تحجب الأخطاء بنسبة 47%، وارتفع معدل فقدان الشيفرة (churn) في غضون أسبوعين بنسبة 15%. فرغم أن إنتاج الشيفرة ارتفع فعليًا ارتفاعًا كبيرًا، فإن الدين التقني المتراكم معه لا يقل خطرًا، وهو يتركز بالأساس عند المطورين الذين لم يدركوا بعد أن بعض أنماط كتابة الشيفرة تهدد قابلية الصيانة الطويلة الأمد للمستودع.

أما عن الدين التقني، فتقول الورقة «^[8]Debt Behind the AI Boom» إن «رائحة الشيفرة هي أكثر أنواع المشكلات شيوعًا، إذ تبلغ نسبتها 89.3%»، و«من بين الإيداعات التي أنتجتها مختلف مساعدات البرمجة بالذكاء الاصطناعي، أدخل أكثر من 15% منها مشكلة واحدة على الأقل. والأهم من ذلك أن 22.7% من

المشكلات التي أدخلها الذكاء الاصطناعي لا تزال قائمة في أحدث إصدار من مستودع الشيفرة». و«تدل هذه النتائج على أن الشيفرة المولدة بالذكاء الاصطناعي قد تحمّل مشروعات البرمجيات الفعلية تكاليف صيانة طويلة الأمد، كما تُبرز ضرورة تعزيز ضمان الجودة في التطوير بمساعدة الذكاء الاصطناعي».

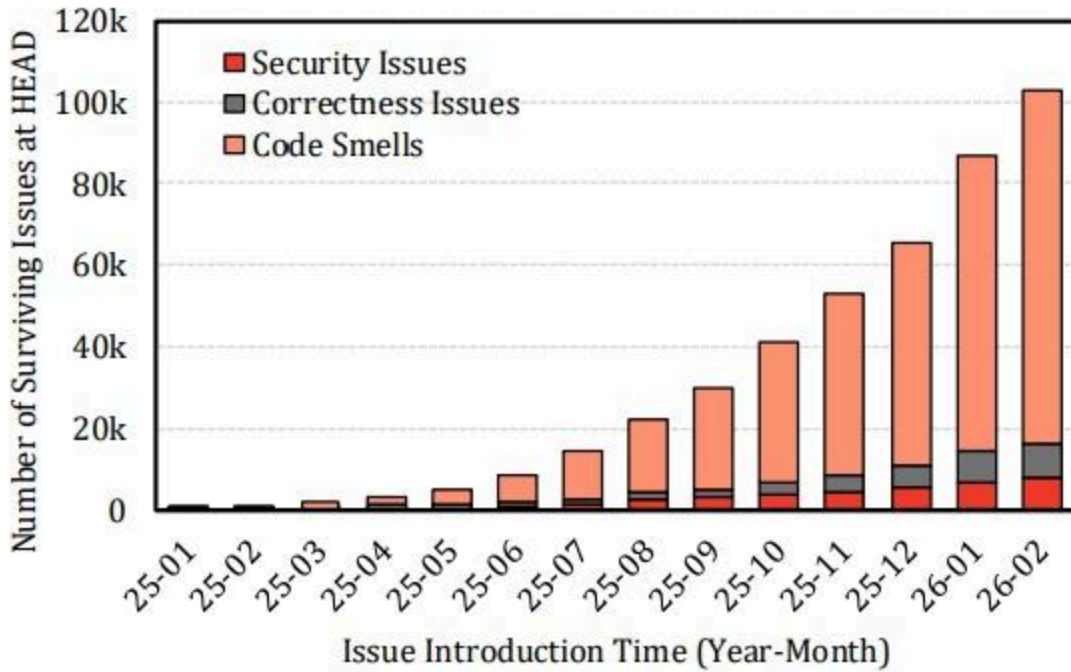


Fig. 9: Cumulative growth of AI-introduced issues over time, by issue type.

المصدر: «Debt Behind the AI Boom»

وأما ورقة «[9] Faster Code, Deeper Debt?» فتقول إن «نماذج LLM كثيرًا ما تزيد من حدة الأشكال التقليدية للدين التقني (ولا سيما الدين في الشيفرة والتصميم والتوثيق)، وتستحدث في الوقت نفسه أنواعًا جديدة من الدين خاصة بها. ومن الجدير بالملاحظة أننا نعرفنا على نوع من «دين الدمج السريع»، أي التضحية بالجودة طلبًا للسرعة، فتؤدي الشيفرة المولدة بسرعة إلى سلسلة من ردود الفعل، تولّد دينًا في الحكامة وتزيد تكاليف الصيانة على المدى الطويل. ومن الفئات الناشئة الأخرى للدين دين التوجيهات (prompt)، والدين الأخلاقي، والدين البياني، ودين التتبع، وهي تعكس التحديات الفريدة التي تطرحها تطبيقات LLM».

لا تؤدي البرمجة بالذكاء الاصطناعي إلى تراكم الدين التقني فحسب، بل تجلب ديونًا أخرى أيضًا. فقد أشارت عالمة الحاسوب Margaret-Anne Storey في ورقتها «From Technical Debt to Cognitive and Intent Debt» [10] إلى أمرين: أولهما «الدين المعرفي» (cognitive debt)، أي ضياع التوافق داخل الفريق؛ وثانيهما «دين النوايا» (intent debt)، أي غياب الأساس المنطقي المُخرَج إلى الخارج الذي يحتاجه المطورون ووكلاء الذكاء الاصطناعي للتعامل الآمن مع الشيفرة. وبكلام أوضح: قد يؤدي

الاستخدام الشامل للبرمجة بالذكاء الاصطناعي إلى ألا يعرف أحد في الفريق كله ما الذي يفعله . وتركيز القطاع اليوم لا يزال على الدين التقني، أما هذه المسائل الأعمق فلم تنكشف بعد انكشافًا كاملاً .

وبعد كل هذا، أظن أن القارئ الكريم قد بلغ الجواب في نفسه . لقد كررت في إجابات كثيرة (مثل: هل تعتقد أن البرمجة بالذكاء الاصطناعي ستحل حقًا محل المبرمجين في المستقبل؟ وكيف ننظر إلى توقع مسؤول الذكاء الاصطناعي في مايكروسوفت بأن «معظم وظائف الياقات البيضاء سيستبدلها الذكاء الاصطناعي استبدالًا كاملاً خلال 18 شهرًا»؟) رأيًا واحدًا: سيغيّر الذكاء الاصطناعي طريقة عمل ذوي الياقات البيضاء تغييرًا عميقًا، لكنه لا يستطيع أن يُقصي جميع المبرمجين، وأحرى أن يُقيل جميع ذوي الياقات البيضاء . وقد أثبتت الوقائع الفعلية صحة رأيي .

ومع تبدد أسطورة أن الذكاء الاصطناعي سيحل محل كل ذوي الياقات البيضاء، بدأت تظهر في سوق رأس المال مؤشرات على انفجار الفقاعة . وقد استنتجت سابقًا، على افتراض رفع كفاءة تطوير البرمجيات بنسبة 30%، أن فقاعة الذكاء الاصطناعي موجودة فعليًا وأن حجمها أكبر من فقاعة الإنترنت (انظر: «ما حجم فقاعة الذكاء الاصطناعي؟ وهل ستتجاوز أزمته أزمة 2008 المالية؟») . وأما حديث مدراء شركات النماذج الكبيرة عن «دمار الذكاء الاصطناعي للبشرية» في الآونة الأخيرة، فهو ليس إلا جنون اللحظة الأخيرة قبل الفناء .

المراجع

- The AI revolution in software development <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>
- Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools. https://download.ssrn.com/2026/9/6/6843118.pdf?response-content-disposition=inline&X-Amz-Security-BGEXoeayHvn1JN4NCfFjioMOP%2B8czxCNN1WRk1xigrdJae4Ew52iInuVz%2FDmizuSzgvtww9A%3D%3D&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Date=20260919T025315Z&X-Amz-SignedHeaders=host&X-Amz-Expires=300&X-Amz-Credential=ASIAUPUUPRWEYZTNLYVN%2F20260919%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-e=4ba6ead523ab5e286d4d945a3af8b9fbf80ada4d1e8f200c179fede48d0732b3&abstractId=6843118
- On AI Coding and Its Discontents <https://calnewport.com/on-ai-coding-and-its-discontents>
- I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25

Generation <https://arxiv.org/pdf/2605.06445>

Coding <https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-/enterprise-ai-coding>

The Maintainability Gap: AI Code Quality in 2026

https://www.gitclear.com/the_ai_code_quality_maintainability_gap

Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild <https://arxiv.org/abs/2603.28592>

Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and
Its Early Signs in LLM-Assisted Software
Development <https://arxiv.org/abs/2606.14796>

From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI <https://arxiv.org/abs/2603.22106>

الحواشی

The AI revolution in software development ^[1]

中文原文: The AI revolution in software development

<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>

Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools [2]

中文原文: Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools

https://download.ssrn.com/2026/9/6/6843118.pdf?response-content-disposition=inline&X-Amz-Security-Token=IQoJb3JpZ21uX2VjEiR%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2FwEaCXVzLWVhc3QtMSJHMEUCIA4K1xaTLIbYG81%2B4GGUGWJIOP1bXefV1tiMEDrjOT7UaIEAxlaBFnksn8BvTmtJugDcWK1imIcsg%2BFtjq05fZmGv3gqvQUIUxAEGgwzMDgONzUzMDeyNtciDL6mjhl5IYItgnLyySqaBXzW0%2BLuUelPo7ACCP5mwJ4tKLHkJ7VxcmWECyylj%2B18zQrjPZ9d9oVhTucj3AeO8TSgt%2FdBwQKaRdz%2FPW5X9g8ck8FzsJYPV5ADNNYitwvTqRPH0tmwIvdANXIo2MmE%2Fq0tUxp8r7dYnLkml17nao%2Fo0bC0uBU5bFDtp8z8L6peJLcFA1xYV%2FeYRQkwubHG%2FWCr4xxKPH0z%2FAqzs2Fy%2FCZV11TA38%2BRZ7CfsFrB8LbWKaXFztjW2hGat4LtLubJMYIsDjew4S8jxmJAh7AeyBkh7DLufKHDnWBDozx%2F1aJycLzA7KKd6EnmSVjIa8HNACI4sibNpVZcfiFtbAqvQmoiqc1B%2B2RHggWcbS9ZA8dRigETQmzcZSV3GJkKiYDOGIOPIBI59uitj6R54orFsestnCMONvNvRXEtk4HOHZT3aTFC1D6D4dcQZCLB5VltvBF6dwzTpxT4SKagfjfenKolISfu5Jzf6Vb2SZJfU9xZxJWESIVQGru%2BkAhJyAni%2BNmo7ZzX%2B0cbgjUbGn5J4q47zi41KMrhsGDamp1ALeQtDwTmTYhs70fRNF6%2Fq6ix52SBYcWt8QuhUtagYDmm6%2FaGGOVkASEbmW8hoOI2chBQeJBdUegB9NGgplbFydPSyX9WI2DsOGZMcJTlZHcjYia6tbSSwKF4jICpBFPXyMf33i0bhQWRjNXunDOBE3UFyI6Fzm%2BG%2BBE0mDrNVwYATycOGXvr9v2KbTR6HjWEZCUHgeKLevjTbrDzjNXu1%2FgkIwJS7t7IWPPgGCMesvEl ex9QMfDi945ep58TXzu61kXHeZA2rk56RcIUEEc71bnV%2Bs2BibiYEWiSiXfd4TNFEMehOwdLpS0pRQNN5MJ4zu6sjTGE1S6L1j%2FTC6zbfVBjqxAb6xSZFcMcJuzZhj7V%2B46YkBBPIGM5omj16jenI1gv2E%2BHRp9t11%2B6Dt5A8oetYaA8S%2FCI8nfSGx8bUk8%2FRZbediBWTeAYfyta39CDTT

GZzqWGX4Zg2FYmVySCcUFJL%2B%2Fi8AyosppJMVPwCI26CSJquHPD54%2BLssBLEFZNBGEXoeayHvn1JN4NCffjioMOP%2B8czxCNN1WRk1xigrdJa
e4Ew52iInuVz%2FDmizuSzgvtww9A%3D%3D&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Date=20260919T025315Z&X-Amz-SignedHeader
s=host&X-Amz-Expires=300&X-Amz-Credential=ASIAUUPRWEYZTNYLVN%2F20260919%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Sig
nature=4ba6ead523ab5e286d4d945a3af8b9fbf80ada4d1e8f200c179fede48d0732b3&abstractId=6843118

On AI Coding and Its Discontents [3]

中文原文: On AI Coding and Its Discontents
[/https://calnewport.com/on-ai-coding-and-its-discontents](https://calnewport.com/on-ai-coding-and-its-discontents)

I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth [4]

中文原文: I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth
https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25

Constraint Decay: The Fragility of LLM Agents in Backend Code Generation [5]

中文原文: Constraint Decay: The Fragility of LLM Agents in Backend Code Generation
<https://arxiv.org/pdf/2605.06445>

The Productivity Illusion at the Heart of Enterprise AI Coding [6]

中文原文: The Productivity Illusion at the Heart of Enterprise AI Coding
[/https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding](https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding)

The Maintainability Gap: AI Code Quality in 2026 [7]

中文原文: The Maintainability Gap: AI Code Quality in 2026
https://www.gitclear.com/the_ai_code_quality_maintainability_gap

Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild [8]

中文原文: Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild
<https://arxiv.org/abs/2603.28592>

Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs [9]

in LLM-Assisted Software Development

中文原文: Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early
Signs in LLM-Assisted Software Development
<https://arxiv.org/abs/2606.14796>

From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI [10]

中文原文: From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age
of AI
<https://arxiv.org/abs/2603.22106>

نُشر هذا المقال أصلاً على موقع تشيهو (Zhihu) بالصينية: <https://zhuanlan.zhihu.com/p/2084561490930045240>

المؤلف: 最最遥远的路 • صفحة تشيهو: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>

本文原发表于知乎专栏/回答 (中文): <https://zhuanlan.zhihu.com/p/2084561490930045240>; 作者: 最最遥远的路: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>